

1. What are the most challenging aspects of the coursework task?

The most difficult part in this coursework was to introduce clean and modular logic from the start of the work while maintaining complete Pylint compatibility. Getting the balance right between functional gameplay and clean coding practices such as well-structured functions, appropriate naming and elimination of wildcard imports took a lot of attention. Designing the computer's decision making for optimal or defensive moves was also a challenge as well having to make sure it can block the user or restrict its victory. In addition to managing board state, user input validation and smooth interaction without crashes or bugs required a clean understanding of control flow and error handling. Adding that layer of complexity of integrating file I/O in the leaderboard but still being able to maintain data persistence made things even more complex.

2. How did you go about completing the task?

I started with gaining the notion of the needed function templates and with making sure that their signatures were not altered as was required. I initially added the game's basic mechanisms (the setting of the board, determining the moves, and checking win conditions), then expanded to implement computer logic with almost no clever heuristics. Once I had a working version of the game I started working on the leaderboard, which is stored and retrieved as JSON. After the gameplay, and leaderboard behaved as I wanted them to, I then decided to refactor, such as Pylint, code style, like removing wildcard imports or adding docstrings. I tested each module separately and also combined both files to perform the complete flow through `play_game.py`. The last step was to debug and tinker with the user interface for clarity and interaction before validation.

3. What have you learned over the course of completing this coursework task?

This coursework has helped me to learn how to develop a complete, operational Python application using modular principle and clean code as its practices. I learnt more on control structures, nested loops and input validation. Reinforcing algorithmic thinking was supported by implementing win/draw logic and basic AI logic for the computer. I also learned the importance of writing reusable, documented functions and how to follow the standard of coding such as PEP 8 and Pylint guidelines. Using file handling and JSON serialization as part of the exercise taught me about persistent storage and how to keep information on a user's session. Lastly I acquired important experience in debugging, testing, and fine-tuning of writing of code to develop a User-friendly command line interface.